# Efficient Test-Time Scaling for LLM-based Time Series Forecasting


Xuan-May Le*
xuanmay.le@student.unimelb.edu.au
The University of Melbourne
Melbourne, Victoria, Australia

Minh-Tuan Tran*
tuan.tran7@monash.edu
Monash University
Melbourne, Victoria, Australia

Ling Luo
ling.luo@unimelb.edu.au
The University of Melbourne
Melbourne, Victoria, Australia

Uwe Aickelin
uwe.aickelin@unimelb.edu.au
The University of Melbourne
Melbourne, Victoria, Australia

Dinh Phung
dinh.phung@monash.edu
Monash University
Melbourne, Victoria, Australia

Trung Le
trunglm@monash.edu
Monash University
Melbourne, Victoria, Australia


arXiv:2608.08675v1 [cs.LG] 9 Aug 2026


## Abstract

Long-term time series forecasting benefits from preserving global structure such as trends and seasonality. Recent LLM-based forecasters often improve accuracy through test-time scaling (e.g., iterative refinement), but these methods are computationally expensive and increasingly prone to global-shape mismatch as the prediction horizon extends. We propose SCALER, a coarse-to-fine forecasting framework that first employs a lightweight Transformer tailored to long-term shape modeling to predict a coarse representation of future dynamics. This predicted shape then serves as a compact guide for an LLM to perform test-time scaling via iterative coarse-to-fine residual token refinement, while processing substantially fewer tokens at each step. By guiding refinement with an explicit future-shape prediction, SCALER reduces reliance on long description prompts, and its fixed-step refinement avoids costly reward-model-based selection, further lowering computational overhead. Experimental results demonstrate that SCALER outperforms strong forecasting baselines in long-term, short-term and zero-shot forecasting while significantly reducing the inference cost associated with scaled LLM for time series forecasting. Code: https://github.com/xuanmay2701/SCALER.



## CCS Concepts

• **Information systems** → **Data mining**; • **Computing methodologies** → **Machine learning**.

## Keywords

time series; time series forecasting, test-time scaling, reasoning, large language model, llm

**ACM Reference Format:**

Xuan-May Le, Minh-Tuan Tran, Ling Luo, Uwe Aickelin, Dinh Phung, and Trung Le. 2026. Efficient Test-Time Scaling for LLM-based Time Series Forecasting. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26), August 09–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3770855.3818201


*Both authors contributed equally to this research.


This work is licensed under a Creative Commons Attribution 4.0 International License.


*KDD '26, Jeju Island, Republic of Korea*


© 2026 Copyright held by the owner/author(s).


ACM ISBN 979-8-4007-2259-2/2026/08
https://doi.org/10.1145/3770855.3818201

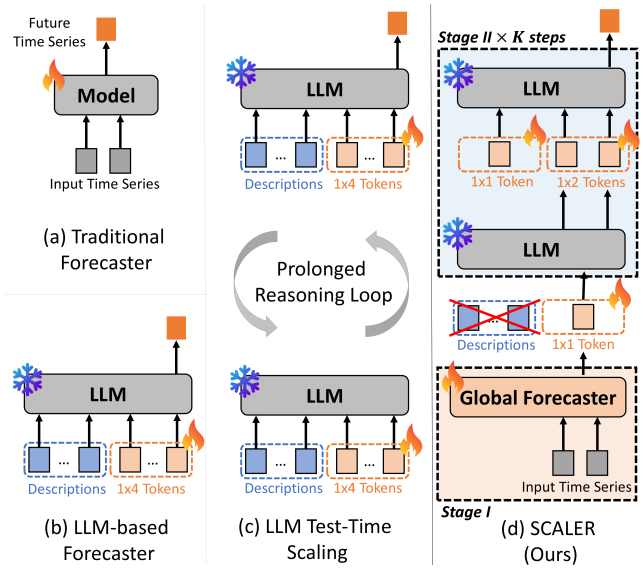




**Figure 1: (a) Standard: direct forecasting; may lose long-horizon trend/seasonality. (b) LLM forecaster: prompt-based prediction; long horizons need long prompts and can drift in shape. (c) LLM + test-time scaling: long iterative refinement improves accuracy but is costly and can amplify shape mismatch. (d) SCALER (ours): predicts coarse future shape first, then undertakes fixed-step residual refinement with fewer tokens for cheap, shape-consistent forecasts.**

## 1 Introduction

Time series data consists of sequential observations indexed over time [2, 12–14, 27, 33, 35], and forecasting such sequences plays a central role in many real-world domains, such as finance [26, 41], healthcare [10, 25], weather prediction [8, 39], and energy demand modeling [24, 29], where reliable future estimates directly support decision-making and resource allocation. However, many conventional forecasting approaches face difficulties capturing complex temporal dynamics at multiple scales, often failing to jointly represent long-range dependencies and fine-grained local variations in the data.

Over the past decades, numerous approaches have been proposed for time series forecasting. Traditional deep-learning-based models [20, 28, 34, 36, 37] (Figure 1a) are effective at capturing global temporal patterns such as trends and seasonality through

nonlinear architectures, and achieve strong performance on standard benchmarks. More recently, large language model (LLM) forecasters [9, 17, 31, 32, 43] (Figure 1b) have attracted renewed interest, bringing multimodal understanding and generative modeling perspectives to time series forecasting. Despite these advances, most LLM-based TSF methods still formulate forecasting as direct sequence-to-sequence mapping, producing predictions in a single inference pass without explicit intermediate reasoning. Consequently, scaling-time inference strategies [4] (Figure 1c) have been explored to improve accuracy by generating multiple candidates or iteratively refining predictions, but they face two key challenges: (1) iterative refinement and candidate selection can be computationally expensive, causing inference cost to grow rapidly with horizon length and desirable accuracy; and (2) repeated local corrections can accumulate into a global-shape drift, distorting trends and seasonality as the horizon extends. These issues are often exacerbated by reliance on long prompts or expensive reward-model-based candidate selection.

To address the above issues, we introduce SCALER, a two-stage coarse-to-fine framework (Figure 1d) designed for efficient test-time scaling in long-horizon forecasting. SCALER first tokenises the history into a compact multi-scale representation using a shared encoder, which amortises feature extraction and prevents repeated processing of raw inputs. In Stage I, a lightweight forecaster predicts an explicit coarse future shape that captures the horizon's low-frequency structure (trend, seasonality, and regime changes). In Stage II, we allocate additional test-time computation through a fixed K-step refinement procedure driven by a pretrained LLM. The LLM's pretrained generative prior improves robustness and coherence when refining under limited supervision. Conditioned on the history tokens and the coarse-shape anchor, it generates finer-scale token blocks that progressively enrich local details without deviating from the intended global trajectory. The final forecast is produced by decoding the refined token context back to the original resolution. This design scales inference with a fixed, deployment-friendly budget, avoids reward-model-based candidate selection, and mitigates global-shape mismatch that becomes more severe as the horizon grows.

This design provides two practical advantages. **(i) Efficiency:** SCALER offloads global-structure prediction to a lightweight forecaster and uses the LLM only for incremental refinement over compact token blocks, substantially reducing the token-processing burden of scaled inference. In avarge inference time, our SCALER is 7 times faster than standard test-time scaling forecasters. **(ii) Stability:** anchoring refinement to an explicit coarse shape constrains the LLM's updates, reducing reliance on long prompts and limiting long-horizon drift. Moreover, the fixed-step refinement schedule yields deterministic test-time computation and avoids costly reward-model-based candidate selection, making inference predictable and deployment-friendly.

Our main contributions are:

- We propose SCALER, a two-stage framework for *test-time scaling* in long-horizon forecasting: a lightweight forecaster predicts a coarse future shape, and a pretrained LLM refines it into full-resolution forecasts.
- We introduce multi-scale tokenization with a *fixed-step* coarse-to-fine refinement schedule, where each scaling step generates only a compact token block conditioned on the shape anchor, reducing computational cost and limiting long-horizon drift.
- Experiments across long-term, short-term, and zero-shot benchmarks show that SCALER improves accuracy over strong baselines while cutting the inference cost compared to standard test-time scaling LLM-based time series forecasters.

## 2 Related Work

**Time series forecasting.** A wide range of TSF methods span statistical, deep learning, and generative paradigms. Classical models such as ARMA [27] and ARIMA [2] mainly capture linear dependencies, while Transformer variants including Informer [45], Autoformer [38], Crossformer [44], and Pyraformer [19] use attention to model long-range temporal structure. PatchTST [28] improves efficiency by patchifying sequences, and iTransformer [20] captures multivariate interactions via transposed attention. Lightweight linear baselines (e.g., DLinear [42], RLinear [16], TimeMixer [34], TimeXer [36]) remain competitive on large benchmarks. Overall, these deterministic forecasters are typically effective at preserving coarse global dynamics (trends/seasonality), which becomes increasingly important for long horizons.

Generative TSF is another active direction. Diffusion-style methods such as TimeGrad [30], D3VAE [15], and TSDiff [11] synthesise futures via iterative generation, and ARMD [6] predicts future chunks with an autoregressive moving-window strategy. While such models can better recover local variability and uncertainty, iterative generation can accumulate errors and drift in global structure over long horizons [11, 15, 30], motivating approaches that explicitly anchor long-term shape while refining details.

**LLM-based and foundation-model forecasting.** LLM-based TSF has grown rapidly, bringing multimodal understanding and generative modeling perspectives into forecasting. TimeLLM [9] reprograms time-series inputs to match a pre-trained LLM interface, PromptCast [40] uses prompt-driven textualisation, LLMTime [7] studies numeric token encodings, and AutoTimes [21] aligns forecasting with next-token prediction. Other methods adapt parameters for better temporal alignment, including fine-tuning (FPT [46], LLM4TS [3]) and lightweight updates (CALF [18]), while LVICL [43] strengthens ICL by injecting learned vectors. However, most LLM-based approaches still treat forecasting as direct sequence mapping (often in a single pass) and rely on long prompts or adaptation pipelines that increase compute/memory. Our method instead keeps LLM parameters frozen and targets efficient, shape-faithful long-horizon forecasting.

**Test-time scaling and iterative refinement.** Recent "slow-thinking" inference improves TSF by scaling test-time compute, e.g., sampling multiple futures, selecting candidates, or iteratively refining predictions [4]. While effective, these strategies face two key issues for TSF: (1) cost grows quickly with horizon length because each round reprocesses long sequences and may require expensive selection (often via reward models); and (2) repeated local edits can compound into global-shape drift, distorting trends and seasonal phase as horizons extend. These limitations motivate our decoupled design that anchors refinement to an explicit long-horizon structure while enabling efficient fixed-step scaling.

# 3 Proposed Method

## 3.1 Problem Setup

We study multivariate forecasting. Given a historical sequence with $C$ variables, $X = \{x_t\}_{t=1}^{L}$ and $x_t \in \mathbb{R}^C$, our goal is to predict the future horizon $Y = \{y_t\}_{t=L+1}^{L+T}$. In that, $L$ and $T$ are the lengths of history and future time series, respectively. We seek forecasts that preserve *global structure* (e.g., trends, seasonality, regime shifts) while recovering *local details* under a limited test-time budget.

## 3.2 Overview of SCALER

SCALER is a coarse-to-fine framework with two stages (refer to Figure 2).

**Stage I: Coarse-shape prediction.** A lightweight forecaster $\mathcal{F}$ predicts a downscaled future shape token $\hat{S}^1$ that summarises low-frequency dynamics over the full horizon.

**Stage II: Fixed-step multi-scale refinement.** We encode the history into tokens $Z_I = \mathcal{E}(X)$ and initialise the conditioning context as $Z_S = [\hat{S}^1]$. A refiner $\mathcal{G}$ then performs a fixed $K$-step residual refinement from coarse to fine: at step $k$, it predicts a finer-scale token block $S^k$ conditioned on $(Z_I, Z_S)$, and appends it to $Z_S$ for the next step. This yields a shape-faithful refinement procedure with predictable inference cost and no candidate selection.

## 3.3 Multi-Scale Patch Encoding

We encode each time series into a sequence of *embedding tokens*.

**Encoding.** Given a multivariate series $X \in \mathbb{R}^{L\times C}$, we patchify the input and encode it into a history token sequence $Z_I = [I_1, \ldots, I_m]$:

$$Z_I = [\, I_1, \ldots, I_m \,] = \text{Encoding}(X) = \mathcal{E}(\text{Patching}(X; \ell)), \quad (1)$$

where $\ell$ is the patch length, $m$ is number of tokens, $\mathcal{E}$ is a patch encoder that aligns time-series patches with the LLM embedding space via prototype-based cross-attention. We construct a compact prototype set by linearly projecting the LLM word-embedding matrix $W$,

$$P = \text{Linear}(W), \quad (2)$$

and encode each patch by cross-attending its query (from patch token $I$) to prototypes $P$ as keys and values:

$$Z = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V, \quad (3)$$

where $Q$ is projected from $I$, and $K, V$ are projected from $P$.

**Downscaling.** Let $\{\text{Scale}_k\}_{k=1}^{K}$ be ordered from coarse (e.g. 12 and 24 timestamps) to fine (e.g. 48 and 96 timestamps). For the future horizon $Y \in \mathbb{R}^{T\times C}$, we optionally construct a downscaled view at scale $k$ via a normalise-then-interpolate operator:

$$Y^k = \text{Downscaling}(Y; \text{Scale}_k) \quad (4)$$

$$= \text{Down-interpolating}(\text{Norm}(Y),\ \text{Scale}_k)\,, \quad (5)$$

where $\text{Norm}(\cdot)$ normalises each channel (e.g., by mean and std), and $\text{Down-interpolating}(\cdot)$ downscales the sequence by interpolating to the target resolution (e.g., linear interpolation). We then obtain the token sequence by the same patch-encoding operator:

$$Z_S = [\, S_1^k, S_2^k, \ldots, S_{n_k}^k \,] = \text{Encoding}(Y^k), \qquad S_j^k \in \mathbb{R}^D, \quad (6)$$

where $n_k$ is the number of patches (i.e., tokens) obtained after patching $S^k$.

For multivariate inputs we apply this per channel and concatenate token streams across channels with lightweight identifiers to form the final conditioning sequence.

## 3.4 Stage I: Global Coarse-Shape Forecaster

Given the history $X = x_{1:L}$, we first predict an explicit *coarse global future shape* that captures low-frequency dynamics over the horizon. A compact transformer model $\mathcal{F}$ predicts the coarse shape from the history:

$$\hat{S}^1 = \mathcal{F}(X), \quad (7)$$

where $\mathcal{F}$ is intentionally lightweight since it only models global structure (trends/seasonality/regime) at a reduced resolution.

We next construct the coarse target by applying our normalise-then-interpolate downscaling operator to the future segment $Y = I_{L+1:L+T}$:

$$S^1 = \text{Downscaling}(Y; \text{Scale}_1) \quad (8)$$

We then obtain its coarse-shape token sequence by joint patch encoding:

$$Z_{S^1} = [\, S_1^1, S_2^1, \ldots, S_{n_1}^1 \,] = \text{Encoding}(S^1) \quad (9)$$

**Shape supervision.** We train $\mathcal{F}$ with a regression loss against the coarse target:

$$\mathcal{L}_{\text{shape}} = \mathcal{L}_{\text{MSE}}(\hat{S}^1, S^1) = \|\hat{S}^1 - S^1\|_2^2, \quad (10)$$

(with $\mathcal{L}_{\text{MSE}}$ as a drop-in alternative). The predicted coarse shape $\hat{S}^1$ initialises the conditioning context for the subsequent fixed-step coarse-to-fine refinement.

## 3.5 Stage II: Fixed-Step Multi-Scale Refinement

The predicted coarse shape $\hat{S}^1$ captures global structure but omits fine-grained details. We implement *test-time scaling* through a fixed $K$-step coarse-to-fine refinement loop, where additional inference compute is allocated to progressively finer temporal scales. Crucially, using a pretrained LLM $\mathcal{G}$ as the refiner injects a strong sequence prior, which helps produce coherent long-horizon completions and stabilizes refinement when supervision is limited.

**Token initialisation.** We encode the history once to obtain the history token sequence:

$$Z_I = [\, I_1, I_2, \ldots, I_m \,] = \text{Encoding}(X). \quad (11)$$

We initialise the refinement context with the coarse-shape tokens predicted in Stage I:

$$Z_S = [\, S^1 \,], \qquad \text{where } S^1 = \text{Encoding}(\hat{S}^1)). \quad (12)$$

**Test-time scaling policy (multi-scale refinement).** We treat $\{\text{Scale}_k\}_{k=1}^{K}$ as a test-time scaling schedule that determines how refinement process is distributed across resolutions. During training, we form the scale-$k$ target by a normalise-then-interpolate downscaling:

$$S^k = \text{Downscaling}(Y; \text{Scale}_k) \quad (13)$$

We then obtain its token sequence:

$$Z_{S^k} = [\, S_1^k, \ldots, S_{n_k}^k \,] = \text{Encoding}(Y^k). \quad (14)$$

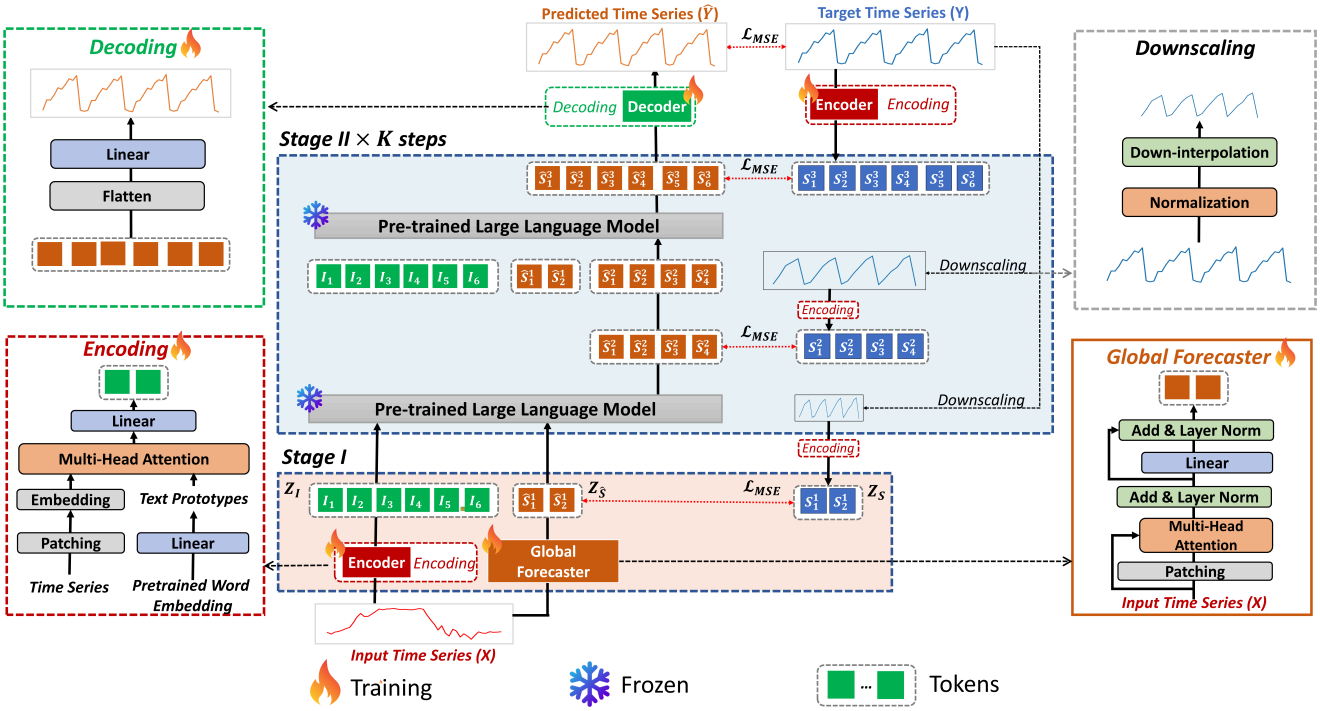



**Figure 2: SCALER architecture. Given the input/history time series $X$, a lightweight coarse forecaster $\mathcal{F}$ predicts a downscaled future shape token $\hat{S}^1$ that captures low-frequency dynamics. A shared multi-scale encoder $\mathcal{E}$ maps $X$ into a continuous token sequence $Z_I$. Conditioned on $Z_I$ and a growing set of shape tokens initialised as $Z_S = [S^1]$, an LLM refiner $\mathcal{G}$ runs a fixed $K$-step coarse-to-fine loop: at step $k$, it predicts a finer-scale token block $S^k$ and appends it to the context $Z_S$ for the next step. The final forecast is obtained by decoding the finest-scale tokens, $S = \mathcal{D}\left(Z_S^{(K)}\right)$, yielding shape-faithful refinement with predictable inference cost and no candidate selection.**

At step $k$, the refiner pretrained large language model $\mathcal{G}$ predicts the scale-$k$ token block conditioned on the history tokens and the coarser context previously predicted:

$$\hat{S}^k = \mathcal{G}(Z_I, Z_S), \tag{15}$$

and we append it to the context for the next step:

$$Z_S \leftarrow [Z_S, \hat{S}^k]. \tag{16}$$

We supervise refinement at two levels. First, we define $\mathcal{L}_{\text{refine}}$ as a scale-wise $\mathcal{L}_{\text{MSE}}$ loss on the downscaled targets at level $k$:

$$\mathcal{L}_{\text{refine}} = \sum_{k=1}^{K} \left\| \hat{S}^k - S^k \right\|_2^2. \tag{17}$$

**Decoding.** Second, we decode the final token set into a full-resolution forecast using a decoder $\mathcal{D}$:

$$\hat{Y} = \text{Decoding}(Z_S) = \mathcal{D}(Z_S), \tag{18}$$

and impose a $\mathcal{L}_{\text{MSE}}$ loss in the original time-series space:

$$\mathcal{L}_{\text{ts}} = \mathcal{L}_{\text{MSE}}(\hat{Y}, Y) = \left\| \hat{Y} - Y \right\|_2^2. \tag{19}$$

The overall training objective is

$$\mathcal{L} = \mathcal{L}_{\text{shape}} + \lambda \mathcal{L}_{\text{refine}} + \beta \mathcal{L}_{\text{ts}}, \tag{20}$$

where $\lambda$ and $\beta$ balance token-level refinement and time-series reconstruction.

**Algorithm 1:** SCALER Training

**Input:** History $X$, future $Y$, patch length $\ell$, scales $\{\text{Scale}_1, \ldots, \text{Scale}_K\}$, refinement steps $K$, weights $\lambda, \beta$
**Output:** Predicted time series $\hat{Y}$

// Stage I: coarse shape forecasting
$S^1 \leftarrow \text{Downscaling}(Y, \text{Scale}_1)$;
$\hat{S}^1 \leftarrow \mathcal{F}(X)$ ;
$\mathcal{L}_{\text{shape}} \leftarrow \|\hat{S}^1 - S^1\|_2^2$ ;
// Stage II: iterative refining
$Z_I \leftarrow \text{Encoding}(X)$ ;
$Z_S \leftarrow [\text{Encoding}(\hat{S}^1)]$ ;
$\mathcal{L}_{\text{refine}} \leftarrow 0$ ;
**for** $k = 1$ **to** $K$ **do**
  $Y^k \leftarrow \text{Downscaling}(Y, \text{Scale}_k)$ $\hat{S}^k \leftarrow \mathcal{G}(Z_I, Z_S)$ ;
  // predict scale-$k$ tokens
  $\mathcal{L}_{\text{refine}} \leftarrow \mathcal{L}_{\text{refine}} + \left\|\hat{S}^k - \text{Encoding}(Y^k)\right\|_2^2$ ;
  $Z_S \leftarrow [Z_S, \hat{S}^k]$ ;
$\hat{Y} \leftarrow \text{Decoding}(Z_S)$ ;
$\mathcal{L}_{\text{ts}} \leftarrow \|\hat{Y} - Y\|_2^2$ ;
$\mathcal{L} \leftarrow \mathcal{L}_{\text{shape}} + \lambda \mathcal{L}_{\text{refine}} + \beta \mathcal{L}_{\text{ts}}$ ;
**return** $\hat{Y}$;

**Inference.** At test time, we first predict $\hat{S}^1 = \mathcal{F}(X)$, initialise $Z_S = [S^1]$, and run the fixed $K$-step loop to obtain the final token set $Z_S$. We then decode $\hat{Y} = \mathcal{D}(Z_S)$ (optionally upsampling

**Algorithm 2:** SCALER Inference

**Input:** History $X$, patch length $\ell$, refinement steps $K$
**Output:** Forecast $\hat{Y}$

// Stage I: coarse shape forecasting
$\hat{S}^1 \leftarrow \mathcal{F}(X)$ ;
// Stage II: iterative refining
$Z_I \leftarrow \text{Encoding}(X)$ ; // history tokens
$Z_S \leftarrow \left[\text{Encoding}(\hat{S})\right]$ ; // init context
**for** $k = 1$ **to** $K$ **do**
  $\hat{S}^k \leftarrow \mathcal{G}(Z_I, Z_S)$ ; // predict finer-scale tokens
  $Z_S \leftarrow [\, Z_S,\ \hat{S}^k \,]$ ; // append for next step
$\hat{Y} \leftarrow \text{Decoding}(Z_S)$ ; // decode final tokens
**return** $\hat{Y}$;

if needed), yielding predictable inference cost without candidate generation or selection. Algorithm 1 gives the training procedure, while Algorithm 2 details the inference objective.

# 4 Experiments

## 4.1 Experimental Settings

**Datasets.** We benchmark SCALER on standard long-horizon multivariate forecasting datasets used by recent LLM-based forecasters, following the TimeLLM evaluation protocol [9]. Specifically, we use the four ETT variants (ETTh1/ETTh2/ETTm1/ETTm2) together with Electricity (ECL), Traffic, Weather, and ILI. For ETT/Weather /ECL/Traffic, we evaluate forecasting horizons $H \in \{96, 192, 336, 720\}$, while for ILI we follow $H \in \{24, 36, 48, 60\}$. We rely on the official training/validation/test partitions and the unified data processing pipeline provided by Time-Series-Library to keep the experimental setup consistent across all methods. Each variable is normalised using statistics computed on the training split. For long-term forecasting, we report MSE and MAE. For short-term forecasting, we report sMAPE, MASE, and OWA, and for zero-shot forecasting we report sMAPE. All metrics are evaluated on the test split, and results are averaged over three runs with different random seeds. The details of datasets are reported in **Appendix A**

**Baselines.** We compare SCALER with two groups of representative methods, covering both standard deep forecasters and LLM-based forecasting approaches.

**(i) Standard forecasters.** We include strong neural forecasting backbones that are widely used on these benchmarks: iTransformer, DLinear, PatchTST, and TimesNet.

**(ii) LLM-based forecasters and test-time scaling.** We further compare against recent LLM-based and foundation-model forecasters: LVICL [43], AutoTimes [21], TimeLLM [9], FPT [46], Chronos [1], and TimesFM [5]. We further compare against the recent test-time scaling for LLM-based time series forecasting: TimeReasoner [4]. All baselines are evaluated with the same data splits, preprocessing, and metrics to ensure fair comparison. When a method involves test-time scaling, we additionally control the inference budget and keep the prediction horizon matched across methods.

**Implementation Details.** We consider each channel separately, and fixed $\lambda$ and $\beta$ to 1, patch length $\ell = 12$, and set Scale $\in T/8, , T/4, , T/2, , T$ for all experiments. All models are implemented in PyTorch and run on NVIDIA A100-80GB GPUs. We optimise with Adam and choose the learning rate from {3e−5, 5e−5, 7e−5, 9e−5}, using MSE as the training objective. Following common practice for multivariate forecasting, we process each channel as an individual series in training and evaluation. We use the standard input lengths for these benchmarks (e.g., 672 for ETT/Weather/ECL/Traffic). The batch size is selected from {192, 256, 1024, 2048}. We train for at most 40 epochs and apply early stopping when the validation loss does not improve for three consecutive evaluations.

## 4.2 Long-Term Forecasting

Table 1 reports long-term forecasting results averaged over four prediction horizons on seven benchmarks. SCALER achieves the best overall accuracy on most datasets, ranking first on **ETTh1**, **ETTh2**, **ETTm2**, **Weather** (MSE), and **Traffic**, and remaining highly competitive on **ECL** where it ties for the best MSE. Compared with LLM-based forecasters (e.g., LVICL, AutoTimes, TimeLLM, FPT, Chronos, and TimeFM), the proposed coarse-to-fine test-time scaling consistently reduces error while keeping the model efficient. The *1st Count* row further summarises this trend, showing that SCALER obtains the largest number of first-place results across all dataset–horizon pairs, highlighting its robustness across diverse domains and forecasting lengths. Full results are reported in **Appendix E**.

## 4.3 Short-Term Forecasting

Table 2 summarises short-term forecasting performance under four scenario groups (Year, Quarter, Month, and Others) using standard M4-style metrics (sMAPE, MASE, and OWA), where lower is better. SCALER consistently achieves the best results across all scenarios and metrics, obtaining the lowest sMAPE and MASE and therefore the lowest OWA in each group. In particular, SCALER improves over the strongest baseline, TimeReasoner, on every scenario, and yields clear margins over LLM-based forecasters (LVICL, AutoTimes, TimeLLM, and FPT) as well as competitive deep forecasting models (DLinear, PatchTST, and TimesNet). Averaged across scenarios, SCALER attains the best overall accuracy (sMAPE = 11.41, MASE = 1.559, OWA = 0.828). The *1st Count* row further confirms this robustness: SCALER ranks first in all 12 scenario–metric combinations, indicating that the proposed test-time scaling strategy generalises well across different temporal granularities and data characteristics.

## 4.4 Zero-shot Forecasting

To examine cross-dataset generalisation without any target-domain adaptation, we further evaluate SCALER in a zero-shot setting on the **M3** [22] and **M4** [23] benchmarks, following the protocol in [43]. Concretely, we consider two transfer directions: **M4→M3** (train on M4 and test on M3) and **M3→M4** (train on M3 and test on M4). For M4→M3, Yearly/Quarterly/Monthly subsets are transferred from the corresponding M4 frequency to the matching M3 frequency, while the M3-Others subset is trained using M4-Quarterly. For M3→M4, we similarly transfer across matched frequencies, and train M4-Others using M3-Monthly. We report sMAPE as the evaluation metric, and all numbers are averaged over five runs with different random seeds, consistent with [43].

**Table 1: Long-term forecasting results (averaged over horizons). We report MSE/MAE on seven benchmarks. Bold denotes the best result and underline denotes the second best within each dataset. The last row reports the number of first-place results across all dataset–horizon pairs. All results are taken from [43], and we rerun TimeReasoner under our setup. All numbers are averaged over five runs with different random seeds.**

| | Test-Time Scaling | | | | LLM-based Model | | | | | | | | | | | | Other Deep Learning Model | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | SCALER (Ours) | | TimeReasoner | | LVICL | | AutoTimes | | TimeLLM | | FPT | | Chronos | | TimeFM | | iTransformer | | DLinear | | PatchTST | | TimesNet | |
| Dataset | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE |
| ETTh1 | **0.376** | **0.408** | 0.403 | 0.417 | 0.381 | 0.412 | 0.389 | 0.422 | 0.408 | 0.423 | 0.427 | 0.426 | 0.401 | 0.434 | 0.417 | 0.451 | 0.438 | 0.45 | 0.423 | 0.437 | 0.413 | 0.431 | 0.458 | 0.45 |
| ETTh2 | **0.321** | **0.373** | 0.348 | 0.389 | 0.326 | 0.376 | 0.352 | 0.395 | 0.354 | 0.393 | 0.353 | 0.391 | 0.327 | 0.377 | 0.334 | 0.385 | 0.382 | 0.414 | 0.431 | 0.446 | 0.33 | 0.379 | 0.414 | 0.427 |
| ETTm1 | 0.324 | 0.374 | 0.344 | 0.373 | 0.328 | 0.378 | 0.332 | 0.38 | 0.35 | 0.378 | 0.366 | 0.382 | **0.322** | **0.371** | 0.341 | 0.393 | 0.37 | 0.399 | 0.357 | 0.378 | 0.351 | 0.38 | 0.4 | 0.406 |
| ETTm2 | **0.234** | **0.302** | 0.245 | 0.311 | 0.239 | 0.306 | 0.243 | 0.31 | 0.254 | 0.314 | 0.265 | 0.315 | 0.245 | 0.313 | 0.242 | 0.309 | 0.272 | 0.331 | 0.267 | 0.333 | 0.255 | 0.315 | 0.291 | 0.333 |
| ECL | **0.154** | **0.245** | **0.154** | 0.25 | 0.158 | 0.248 | 0.159 | 0.253 | 0.159 | 0.253 | 0.167 | 0.263 | 0.164 | 0.258 | 0.162 | 0.255 | 0.161 | 0.256 | 0.177 | 0.274 | 0.159 | 0.253 | 0.192 | 0.295 |
| Weather | **0.217** | 0.257 | 0.218 | **0.254** | 0.224 | 0.262 | 0.235 | 0.273 | 0.225 | 0.257 | 0.237 | 0.27 | 0.22 | 0.263 | 0.229 | 0.27 | 0.238 | 0.272 | 0.24 | 0.3 | 0.226 | 0.264 | 0.259 | 0.287 |
| Traffic | **0.367** | **0.256** | 0.382 | 0.259 | 0.37 | 0.26 | 0.374 | 0.264 | 0.388 | 0.264 | 0.414 | 0.294 | 0.376 | 0.264 | 0.383 | 0.269 | 0.379 | 0.272 | 0.434 | 0.295 | 0.391 | 0.264 | 0.62 | 0.336 |
| 1st Count | **6** | **5** | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

**Table 2: Short-term forecasting results. We report sMAPE, MASE, and OWA on four scenario groups (Year, Quarter, Month, Others) and their average, where lower is better. Bold denotes the best result and underline denotes the second best within each row. The last row reports the number of first-place results across all scenario–metric combinations. All numbers are averaged over five runs with different random seeds.**

| Scenario | Category | SCALER (ours) | TimeReasoner | LVICL | AutoTimes | TimeLLM | FPT | DLinear | PatchTST | TimesNet |
|---|---|---|---|---|---|---|---|---|---|---|
| Year | sMAPE | **12.984** | 13.059 | 13.101 | 13.319 | 13.419 | 13.531 | 13.866 | 13.517 | 13.394 |
| | MASE | **2.977** | 2.983 | 2.988 | 2.993 | 3.005 | 3.015 | 3.006 | 3.031 | 3.004 |
| | OWA | **0.764** | 0.771 | 0.775 | 0.784 | 0.789 | 0.793 | 0.802 | 0.795 | 0.787 |
| Quarter | sMAPE | **9.82** | 9.935 | 10.01 | 10.101 | 10.11 | 10.177 | 10.689 | 10.847 | 10.101 |
| | MASE | **1.159** | 1.163 | 1.17 | 1.182 | 1.178 | 1.194 | 1.294 | 1.315 | 1.183 |
| | OWA | **0.872** | 0.882 | 0.882 | 0.89 | 0.889 | 0.897 | 0.957 | 0.972 | 0.89 |
| Month | sMAPE | **12.195** | 12.311 | 12.321 | 12.71 | 12.98 | 12.894 | 13.372 | 14.584 | 12.866 |
| | MASE | **0.916** | 0.93 | 0.934 | 0.934 | 0.963 | 0.956 | 1.014 | 1.169 | 0.964 |
| | OWA | **0.856** | 0.865 | 0.867 | 0.88 | 0.903 | 0.897 | 0.94 | 1.055 | 0.894 |
| Others | sMAPE | **4.577** | 4.742 | 4.751 | 4.843 | 4.795 | 4.94 | 4.894 | 6.184 | 4.982 |
| | MASE | **3.117** | 3.117 | 3.127 | 3.277 | 3.178 | 3.228 | 3.358 | 4.818 | 3.323 |
| | OWA | **0.984** | 0.993 | 0.994 | 1.026 | 1.006 | 1.029 | 1.044 | 1.14 | 1.048 |
| AVERAGE | sMAPE | **11.41** | 11.558 | 11.567 | 11.831 | 11.983 | 11.991 | 12.418 | 13.022 | 11.93 |
| | MASE | **1.559** | 1.571 | 1.573 | 1.585 | 1.595 | 1.6 | 1.656 | 1.814 | 1.597 |
| | OWA | **0.828** | 0.837 | 0.838 | 0.85 | 0.859 | 0.861 | 0.891 | 0.954 | 0.867 |
| | 1st Count | **12** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Table 3 shows that SCALER consistently achieves the best performance in both transfer settings. For M3→M4, it obtains the lowest average sMAPE (12.36), surpassing TimeReasoner (12.40) and LVICL (12.596). For M4→M3, it again ranks first across all categories with an average sMAPE of 12.76, outperforming TimeReasoner (12.975) and LVICL (13.032). The *1st Count* metric further highlights its robustness, with SCALER achieving 9 first-place results, demonstrating effective test-time scaling across datasets and sampling frequencies without retraining.

## 4.5 Computation Time Comparison

We compare inference efficiency together with forecasting accuracy on three standard long-term TSF benchmarks (ETTh1, ECL, and Weather). Following common practice, we report MSE (lower is better) and relative inference time (lower is faster), where SCALER is used as the runtime reference and normalised to 1× on each dataset.

Figure 3 summarises the accuracy–efficiency trade-off by plotting MSE against relative inference time for all methods. Across all datasets, SCALER consistently achieves the lowest (or tied-lowest) MSE while remaining computationally efficient. In particular, SCALER is substantially faster than TimeReasoner (about 6.7×–6.8× under the same normalisation) and also yields better accuracy, indicating that heavy test-time scaling can be avoided without sacrificing performance. Compared with faster baselines such as TimeLLM and PatchTST, SCALER provides a clear accuracy gain (lower MSE) at a modest additional inference cost. Overall, these

**Table 3: Zero-shot forecasting results on M3 [22] and M4 [23]. We follow the evaluation protocol of [43] and report sMAPE (lower is better) under two transfer settings: M3→M4 and M4→M3, evaluated on four scenario groups (Year, Quarter, Month, Others) and their average. Bold indicates the best result and underline indicates the second best within each row. The last row reports the number of first-place results across all scenario-category pairs. We report sMAPE as the evaluation metric, and all numbers are averaged over five runs with different random seeds.**

| Scenario | Category | SCALER (Ours) | TimeReasoner | LVICL | AutoTimes | FPT | DLinear | PatchTST | TimesNet | FEDformer |
|---|---|---|---|---|---|---|---|---|---|---|
| M3→M4 | Year | **14.892** | 15.037 | 15.11 | 15.71 | 16.42 | 17.43 | 15.99 | 18.75 | 16 |
| | Quarter | **9.141** | 9.287 | 9.45 | 9.35 | 10.13 | 9.74 | 9.62 | 12.26 | 9.48 |
| | Month | **13.592** | 13.866 | 13.98 | 14.06 | 14.1 | 15.65 | 14.71 | 14.01 | 15.12 |
| | Others | 5.205 | 5.477 | 5.59 | 5.79 | **4.81** | 6.81 | 9.44 | 6.88 | 8.94 |
| | Average | **12.36** | 12.4 | 12.596 | 12.75 | 13.06 | 14.03 | 13.39 | 14.17 | 13.53 |
| M4→M3, | Year | **13.488** | 13.567 | 13.721 | 13.728 | 13.74 | 14.193 | 13.966 | 15.655 | 13.887 |
| | Quarter | **10.504** | 10.66 | 10.734 | 10.742 | 10.787 | 18.856 | 10.929 | 11.877 | 11.513 |
| | Month | **14.299** | 14.498 | 14.56 | 14.558 | 14.63 | 14.765 | 14.664 | 16.165 | 18.154 |
| | Others | **5.912** | 6.089 | 6.219 | 6.259 | 7.081 | 9.194 | 7.087 | 6.863 | 7.529 |
| | Average | **12.76** | 12.975 | 13.032 | 13.036 | 13.125 | 15.337 | 13.228 | 14.553 | 15.047 |
| | 1st Count | **9** | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |

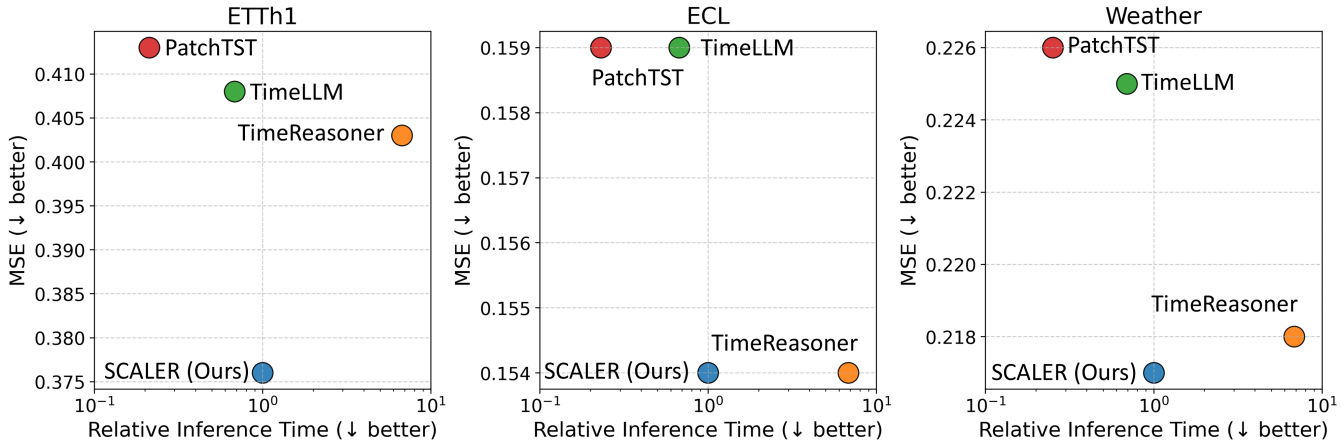



**Figure 3: Accuracy–efficiency trade-off. MSE (y-axis, lower is better) versus relative inference time (x-axis, lower is faster) on ETTh1, ECL, and Weather. SCALER consistently achieves the lowest (or tied-lowest) MSE while remaining computationally efficient. It is much faster than TimeReasoner (about 6.7×–6.8× under the same normalisation) and yields better accuracy, while also improving over TimeLLM and PatchTST in MSE with only modest extra inference cost.**

results place SCALER on a favourable Pareto frontier, demonstrating improved forecasting quality without compromising practical inference efficiency.

## 4.6 Ablation Studies

*4.6.1 Global Forecaster.* We evaluate the benefit of predicting global structure before refinement under two settings:

- **(i) w/o Global Forecaster:** remove $\mathcal{F}$ and feed only the history tokens $Z_I$ to the pretrained LLM refiner.
- **(ii) Global Forecaster (Full Tokens):** use $\mathcal{F}$ to predict full-resolution future tokens, then refine them with the same pretrained LLM.

As shown in the first two method groups of Table 4, introducing a global forecaster consistently reduces both MSE and MAE across datasets and horizons, confirming that explicit global-shape modeling is beneficial. However, forecasting *full-resolution* tokens is substantially more expensive and does not yield better accuracy. We attribute this to the role of the global forecaster: it models low-frequency structure well, but its fine-grained predictions can be unreliable, which may mislead the LLM during refinement. In contrast, SCALER predicts only a downscaled coarse shape and applies fixed-step refinement, achieving the best (or near-best) accuracy with much lower cost.

*4.6.2 Scaling Techniques.* Next, we study scaling-time inference strategies that aim to improve accuracy by increasing test-time compute:

**Table 4: Ablation analysis for our component choices. We report Inference and Training Time normalised by SCALER (SCALER = 1.0).**

| | | Global Forecaster Analysis | | | | Scaling Technique Analysis | | | | Tuning Technique Analysis | | | | | | Default | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | w/o Global Forecaster | | Global Forecaster (Full) | | w/ Description Tokens | | w/ Reward Model | | w/ Prompt Tuning | | w/ LoRA Finetuning | | w/ Full Finetuning | | SCALER (Default) | |
| Dataset | Length | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE |
| ETTh1 | 96 | 0.367 | 0.401 | 0.349 | 0.388 | 0.35 | 0.389 | 0.354 | 0.39 | 0.349 | 0.389 | 0.347 | 0.387 | 0.346 | 0.387 | **0.345** | **0.386** |
| | 192 | 0.408 | 0.423 | 0.381 | 0.41 | 0.378 | 0.411 | 0.381 | 0.413 | 0.378 | 0.41 | 0.376 | 0.408 | 0.375 | 0.407 | **0.374** | **0.406** |
| | 336 | 0.417 | 0.427 | 0.389 | 0.417 | 0.39 | 0.416 | 0.392 | 0.419 | 0.389 | 0.416 | 0.387 | 0.415 | 0.386 | 0.414 | **0.385** | **0.413** |
| | 720 | 0.434 | 0.453 | 0.405 | 0.429 | 0.404 | 0.429 | 0.406 | 0.428 | 0.403 | 0.429 | 0.401 | 0.428 | 0.4 | 0.426 | **0.399** | **0.425** |
| | Avg | 0.407 | 0.426 | 0.381 | 0.411 | 0.381 | 0.411 | 0.383 | 0.413 | 0.38 | 0.411 | 0.378 | 0.41 | **0.375** | **0.406** | 0.376 | 0.408 |
| ECL | 96 | 0.147 | 0.231 | 0.125 | 0.225 | 0.126 | 0.226 | 0.127 | 0.225 | 0.124 | 0.225 | 0.122 | 0.223 | 0.121 | 0.223 | **0.12** | **0.222** |
| | 192 | 0.175 | 0.246 | 0.146 | 0.233 | 0.149 | 0.235 | 0.152 | 0.235 | 0.148 | 0.235 | 0.146 | 0.233 | 0.144 | 0.232 | **0.143** | **0.231** |
| | 336 | 0.188 | 0.252 | 0.165 | 0.248 | 0.163 | 0.251 | 0.166 | 0.25 | 0.161 | 0.25 | 0.16 | 0.249 | **0.156** | **0.245** | 0.158 | 0.246 |
| | 720 | 0.223 | 0.29 | 0.199 | 0.284 | 0.2 | 0.285 | 0.201 | 0.286 | 0.198 | 0.284 | 0.196 | 0.283 | 0.195 | 0.282 | **0.194** | **0.281** |
| | Avg | 0.183 | 0.255 | 0.159 | 0.248 | 0.16 | 0.249 | 0.162 | 0.249 | 0.158 | 0.249 | 0.156 | 0.247 | **0.152** | **0.244** | 0.154 | 0.245 |
| Weather | 96 | 0.162 | 0.212 | 0.145 | 0.196 | 0.146 | 0.197 | 0.149 | 0.196 | 0.144 | 0.197 | 0.143 | 0.196 | 0.142 | 0.195 | **0.141** | **0.193** |
| | 192 | 0.22 | 0.262 | 0.19 | 0.241 | 0.191 | 0.241 | 0.191 | 0.242 | 0.189 | 0.241 | 0.187 | 0.239 | 0.187 | 0.239 | **0.186** | **0.238** |
| | 336 | 0.252 | 0.301 | 0.24 | 0.281 | 0.239 | 0.28 | 0.242 | 0.279 | 0.238 | 0.279 | 0.236 | 0.278 | 0.235 | 0.278 | **0.234** | **0.277** |
| | 720 | 0.329 | 0.346 | 0.31 | 0.323 | 0.311 | 0.322 | 0.314 | 0.324 | 0.309 | 0.324 | 0.308 | 0.322 | **0.305** | 0.321 | 0.306 | **0.32** |
| | Avg | 0.241 | 0.28 | 0.221 | 0.26 | 0.222 | 0.26 | 0.224 | 0.26 | 0.22 | 0.26 | 0.219 | 0.259 | 0.218 | 0.258 | **0.217** | **0.257** |
| Traffic | 96 | 0.362 | 0.258 | 0.339 | 0.246 | 0.34 | 0.249 | 0.34 | 0.25 | 0.337 | 0.248 | 0.336 | 0.247 | 0.335 | 0.246 | **0.334** | **0.244** |
| | 192 | 0.389 | 0.273 | 0.361 | 0.252 | 0.363 | 0.254 | 0.364 | 0.257 | 0.361 | 0.253 | 0.36 | 0.252 | 0.359 | 0.251 | **0.358** | **0.25** |
| | 336 | 0.403 | 0.27 | 0.381 | 0.258 | 0.383 | 0.256 | 0.384 | 0.256 | 0.382 | 0.258 | 0.38 | 0.256 | **0.378** | **0.253** | **0.378** | 0.254 |
| | 720 | 0.423 | 0.286 | 0.404 | 0.28 | 0.403 | 0.281 | 0.405 | 0.279 | 0.403 | 0.279 | 0.401 | 0.278 | 0.4 | 0.277 | **0.399** | **0.276** |
| | Avg | 0.394 | 0.272 | 0.371 | 0.259 | 0.372 | 0.26 | 0.373 | 0.261 | 0.371 | 0.26 | 0.369 | 0.258 | 0.368 | 0.257 | **0.367** | **0.256** |
| Inference Time | | 0.956 | | 1.841 | | 2.341 | | 5.512 | | 1.052 | | 1 | | 1 | | 1 | |
| Training Time | | 0.912 | | 1.724 | | 2.248 | | 5.612 | | 1.211 | | 8.21 | | 15.21 | | 1 | |

- **w/ Description Tokens:** augment the LLM input with additional natural-language description tokens (following prior LLM-forecasting protocols) to provide extra guidance.
- **(w/ Reward Model:** generate multiple candidates and use a reward model to select the best prediction, further increasing inference cost.

The middle method groups in Table 4 show that these strategies provide only marginal (and sometimes inconsistent) gains over the global-forecaster baseline, while noticeably increasing inference time (especially reward-model selection). In contrast, SCALER uses fixed-step refinement and avoids candidate generation/selection, achieving better accuracy with predictable and efficient scaling.

*4.6.3 Tuning Techniques.* Finally, we compare common parameter-efficient and full-parameter tuning strategies for adapting the pre-trained LLM to forecasting:

- **(w/ Prompt Tuning:** learn a small set of soft prompt embeddings while keeping the LLM frozen.
- **(w/ LoRA Finetuning:** insert low-rank adapters and update only adapter parameters.
- **(w/ Full Finetuning:** update all model parameters end-to-end.

As shown in the corresponding method groups of Table 4, tuning can improve performance over pure prompting-based baselines, but it comes with a clear training-time overhead (and full finetuning is the most expensive). In contrast, SCALER achieves the best (or near-best) accuracy without heavy tuning by using a downscaled coarse-shape prior and fixed-step refinement, offering a more efficient accuracy, cost trade-off.

Further ablation studies are reported in **Appendix C**.

## 4.7 Further Analysis

*4.7.1 Additional Results for Different Global Forecaster Backbone Sizes.* To study the impact of global-forecaster capacity, we fix its hidden size to match the LLM backbone (same $d_{\text{model}}$) and vary only the depth from 1 to 4 layers. Figure 4 shows the MSE and MAE on ETTh1 and ECL. Performance is largely stable across depths: deeper models bring only minor, inconsistent gains. Notably, depth= 1 already achieves the best or near-best results, suggesting that matching width (aligned with the LLM) is sufficient to provide an effective coarse shape, while extra depth offers diminishing returns. This supports the use of a lightweight global forecaster for efficient inference without compromising accuracy.

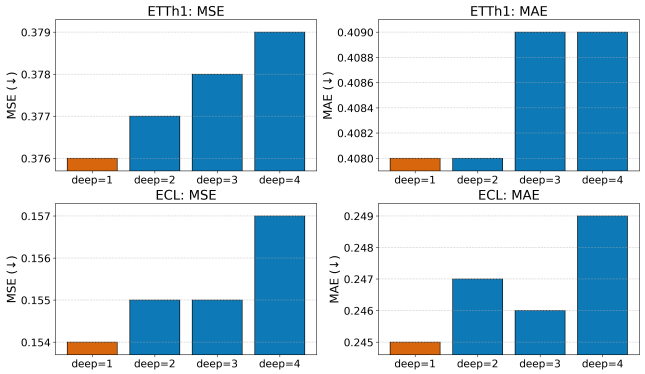



**Figure 4: Effect of the depth of global forecaster. We vary the depth of the lightweight global forecaster from 1 to 4 layers while keeping the hidden dimension $d_{\text{model}}$ fixed to match the LLM backbone. The first setting (depth= 1, orange) is our default configuration, and deeper variants (blue) show only marginal changes in both MSE and MAE on ETTh1 and ECL.**

*4.7.2 Additional Results on Different LLM Backbones.* To test backbone sensitivity, we select the Stage II LLM from LLaMA-7B (default), OPT-2B, OPT-1.3B, OPT-350M, and GPT-2 (124M), while keeping the global forecaster and the refinement pipeline unchanged. Table 5 shows that SCALER remains effective across different families

| Dataset | LLaMA-7B (default) | OPT-2B | OPT-1.3B | OPT-350M | GPT-2 (124M) |
|---|---|---|---|---|---|
| ETTh1 (MSE↓) | **0.376** | 0.379 | 0.381 | 0.381 | 0.379 |
| ETTh1 (MAE↓) | **0.408** | 0.412 | 0.414 | 0.414 | 0.411 |
| ECL (MSE↓) | **0.154** | 0.159 | 0.160 | 0.161 | 0.159 |
| ECL (MAE↓) | **0.245** | 0.253 | 0.256 | 0.259 | 0.254 |

**Table 5: Effect of different LLM backbones on Stage II refinement. SCALER remains effective across a wide range of sizes and families of LLMs, while larger backbones (e.g., LLaMA-7B) consistently yield higher accuracy on ETTh1 and ECL.**

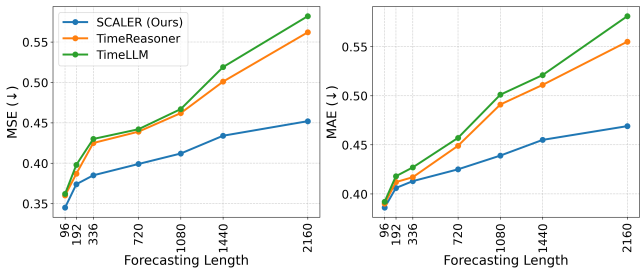



**Figure 5: Long-horizon robustness on ETTh1. MSE/MAE versus forecasting horizon $H \in \{96, 192, 336, 720, 1080, 1440, 2160\}$. SCALER degrades more gracefully as the horizon increases, and the performance gap over TimeReasoner and TimeLLM grows at longer horizons.**

and scales of LLMs: smaller backbones are still competitive, while larger ones consistently improve accuracy. In particular, LLaMA-7B achieves the best performance on both datasets (ETTh1: 0.376 MSE / 0.408 MAE; ECL: 0.154 MSE / 0.245 MAE), indicating that our framework generalises well and supports an accuracy–efficiency trade-off via backbone choice.

*4.7.3 Performance Under Longer Forecasting Horizons.* To further examine robustness as the forecasting horizon increases beyond standard settings, we compare SCALER with TimeReasoner and TimeLLM on progressively longer horizons and report both MSE and MAE. Figure 5 shows a clear trend: while all methods incur larger errors as the horizon grows, SCALER degrades much more gracefully and the advantage becomes more pronounced at longer horizons. In particular, the gap between SCALER and the baselines widens steadily for $H \geq 720$, suggesting that methods without an explicit global-shape anchor are increasingly prone to accumulated mismatch in the long range. In contrast, SCALER maintains a more stable trajectory by first predicting a coarse future shape and then refining details in a coarse-to-fine manner, which better preserves long-term structure. Overall, these results highlight that the proposed shape-anchored refinement is especially beneficial when the forecasting length becomes large, where preserving global dynamics is critical and local-only refinements tend to compound errors.

## 4.8 Shared Encoder Design and Training Stability

SCALER uses a *single shared encoder* $\mathcal{E}$ to tokenise both the input history and the downscaled ground-truth targets at each scale. This design choice is motivated by representation alignment: using a shared encoder ensures that the history tokens $Z_I$ and the scale-$k$ target tokens $Z_{S^k}$ live in the same embedding space, which avoids cross-space mismatch and stabilises training.

While the shared encoder is jointly optimised with the rest of the framework, the multi-task supervision structure—combining $\mathcal{L}_{\text{shape}}$, $\mathcal{L}_{\text{refine}}$, and $\mathcal{L}_{\text{ts}}$—provides complementary gradient signals that collectively prevent overfitting to any single scale. The sensitivity study in Appendix C (Table 7) confirms that SCALER is robust to a wide range of loss weight combinations $(\lambda, \beta) \in [0.5, 2]^2$, which is consistent with stable training dynamics. The consistent performance gains reported throughout Section 4 further support the effectiveness of this design.

## 5 Visualization of Global Shape Drift Mitigation

Figure 6 shows a representative 96-step forecast comparing SCALER against a baseline. The baseline exhibits pronounced global shape drift from step 40 onward, underestimating peak magnitudes and failing to recover the correct oscillatory structure. SCALER closely tracks the ground truth throughout, faithfully reproducing peak locations, trough depths, and the amplitude envelope. This confirms that the proposed method produces structurally coherent predictions rather than merely improving aggregate error metrics.

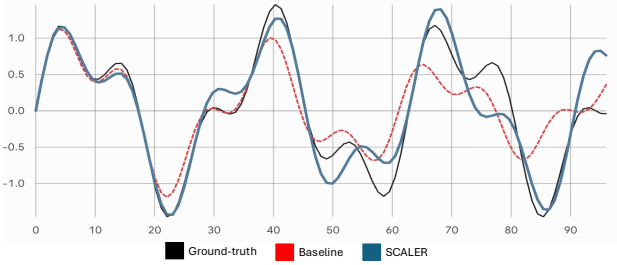



**Figure 6: Representative forecast comparison. Black: ground truth; red dashed: baseline (global shape drift visible from step 40); blue: SCALER (maintains alignment throughout).**

## 6 Conclusions, Limitations and Future Work

We present SCALER, a coarse-to-fine framework that enables *test-time scaling* for long-horizon multivariate forecasting with a pre-trained LLM while preserving global structure. SCALER first predicts an explicit downscaled future shape using a lightweight forecaster, then allocates a fixed $K$-step refinement budget to an LLM refiner conditioned on this shape. This turns additional test-time compute into progressively finer token blocks, producing shape-faithful forecasts with predictable cost and without candidate sampling or reward-model selection. Experiments on standard TSF benchmarks show that SCALER consistently improves accuracy and delivers a stronger accuracy, efficiency trade-off than prior LLM-based test-time scaling approaches.

**Limitations and Future Work.** SCALER currently relies on a fixed refinement schedule, and performance may depend on the LLM backbone and tokenisation strategy. Future work includes adaptive refinement policies, more efficient backbone models, and extensions to challenging real-world scenarios such as missing data and distribution shifts.

## References

[1] Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, et al. 2024. Chronos: Learning the language of time series. *arXiv preprint arXiv:2403.07815* (2024).

[2] Adebiyi A Ariyo, Adewumi O Adewumi, and Charles K Ayo. 2014. Stock price prediction using the ARIMA model. In *2014 UKSim-AMSS 16th international conference on computer modelling and simulation*. IEEE, 106–112.

[3] Ching Chang, Wei-Yao Wang, Wen-Chih Peng, and Tien-Fu Chen. 2025. Llm4ts: Aligning pre-trained llms as data-efficient time-series forecasters. *ACM Transactions on Intelligent Systems and Technology* 16, 3 (2025), 1–20.

[4] Mingyue Cheng, Jiahao Wang, Daoyu Wang, Xiaoyu Tao, Qi Liu, and Enhong Chen. 2025. Can slow-thinking llms reason over time? empirical studies in time series forecasting. *arXiv preprint arXiv:2505.24511* (2025).

[5] Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. 2024. A decoder-only foundation model for time-series forecasting. In *Forty-first International Conference on Machine Learning*.

[6] Jiaxin Gao, Qinglong Cao, and Yuntian Chen. 2025. Auto-regressive moving diffusion models for time series forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 16727–16735.

[7] Nate Gruver, Marc Finzi, Shikai Qiu, and Andrew G Wilson. 2023. Large language models are zero-shot time series forecasters. *Advances in Neural Information Processing Systems* 36 (2023), 19622–19635.

[8] James W Hansen, Simon J Mason, Liqiang Sun, and Arame Tall. 2011. Review of seasonal climate forecasting for agriculture in sub-Saharan Africa. *Experimental agriculture* 47, 2 (2011), 205–240.

[9] Ming Jin, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Y Zhang, Xiaoming Shi, Pin-Yu Chen, Yuxuan Liang, Yuan-Fang Li, Shirui Pan, et al. 2023. Time-llm: Time series forecasting by reprogramming large language models. *arXiv preprint arXiv:2310.01728* (2023).

[10] Shruti Kaushik, Abhinav Choudhury, Pankaj Kumar Sheron, Nataraj Dasgupta, Sayee Natarajan, Larry A Pickett, and Varun Dutt. 2020. AI in healthcare: time-series forecasting using statistical, neural, and ensemble architectures. *Frontiers in big data* 3 (2020), 4.

[11] Marcel Kollovieh, Abdul Fatir Ansari, Michael Bohlke-Schneider, Jasper Zschiegner, Hao Wang, and Yuyang Bernie Wang. 2023. Predict, refine, synthesize: Self-guiding diffusion models for probabilistic time series forecasting. *Advances in Neural Information Processing Systems* 36 (2023), 28341–28364.

[12] Xuan-May Le, Ling Luo, Uwe Aickelin, and Minh-Tuan Tran. 2024. ShapeFormer: Shapelet Transformer for Multivariate Time Series Classification. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1484–1494.

[13] Xuan-May Le, Ling Luo, Uwe Aickelin, Minh-Tuan Tran, David Berlowitz, and Mark Howard. 2025. Ship: A shapelet-based approach for interpretable patient-ventilator asynchrony detection. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining*. Springer, 430–442.

[14] Xuan-May Le, Minh-Tuan Tran, and Van-Nam Huynh. 2022. Learning Perceptual Position-Aware Shapelets for Time Series Classification. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 53–69.

[15] Yan Li, Xinjiang Lu, Yaqing Wang, and Dejing Dou. 2022. Generative time series forecasting with diffusion, denoise, and disentanglement. *Advances in Neural Information Processing Systems* 35 (2022), 23009–23022.

[16] Zhe Li, Shiyi Qi, Yiduo Li, and Zenglin Xu. 2023. Revisiting long-term time series forecasting: An investigation on linear mapping. *arXiv preprint arXiv:2305.10721*.

[17] Chenxi Liu, Qianxiong Xu, Hao Miao, Sun Yang, Lingzheng Zhang, Cheng Long, Ziyue Li, and Rui Zhao. 2024. Timecma: Towards llm-empowered time series forecasting via cross-modality alignment. *arXiv preprint arXiv:2406.01638* (2024).

[18] Peiyuan Liu, Hang Guo, Tao Dai, Naiqi Li, Jigang Bao, Xudong Ren, Yong Jiang, and Shu-Tao Xia. 2025. Calf: Aligning llms for time series forecasting via cross-modal fine-tuning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 18915–18923.

[19] Shizhan Liu, Hang Yu, Cong Liao, Jianguo Li, Weiyao Lin, Alex X Liu, and Schahram Dustdar. 2021. Pyraformer: Low-Complexity Pyramidal Attention for Long-Range Time Series Modeling and Forecasting. In *International Conference on Learning Representations*.

[20] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. 2023. itransformer: Inverted transformers are effective for time series forecasting. *arXiv preprint arXiv:2310.06625* (2023).

[21] Yong Liu, Guo Qin, Xiangdong Huang, Jianmin Wang, and Mingsheng Long. 2024. Autotimes: Autoregressive time series forecasters via large language models. *Advances in Neural Information Processing Systems* 37 (2024), 122154–122184.

[22] Spyros Makridakis and Michele Hibon. 2000. The M3-Competition: results, conclusions and implications. *International journal of forecasting* 16, 4 (2000), 451–476.

[23] Spyros Makridakis, Evangelos Spiliotis, and Vassilios Assimakopoulos. 2018. The M4 Competition: Results, findings, conclusion and way forward. *International Journal of forecasting* 34, 4 (2018), 802–808.

[24] Luis Martín, Luis F Zarzalejo, Jesus Polo, Ana Navarro, Ruth Marchante, and Marco Cony. 2010. Prediction of global solar irradiance based on time series analysis: Application to solar thermal power plants energy production planning. *Solar Energy* 84, 10 (2010), 1772–1781.

[25] Yasuko Matsubara, Yasushi Sakurai, Willem G Van Panhuis, and Christos Faloutsos. 2014. FUNNEL: automatic mining of spatially coevolving epidemics. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*. 105–114.

[26] Abdul Quadir Md, Sanjit Kapoor, Chris Junni AV, Arun Kumar Sivaraman, Kong Fah Tee, H Sabireen, and N Janakiraman. 2023. Novel optimization approach for stock price forecasting using multi-layered sequential LSTM. *Applied Soft Computing* 134 (2023), 109830.

[27] T Mills. 2012. *A very British affair: Six Britons and the development of time series analysis during the 20th century*. Springer.

[28] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2023. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. In *International Conference on Learning Representations*.

[29] Zheng Qian, Yan Pei, Hamidreza Zareipour, and Niya Chen. 2019. A review and discussion of decomposition-based hybrid models for wind energy forecasting applications. *Applied energy* 235 (2019), 939–953.

[30] Kashif Rasul, Calvin Seward, Ingmar Schuster, and Roland Vollgraf. 2021. Autoregressive denoising diffusion models for multivariate probabilistic time series forecasting. In *International conference on machine learning*. PMLR, 8857–8868.

[31] Mingtian Tan, Mike Merrill, Vinayak Gupta, Tim Althoff, and Tom Hartvigsen. 2024. Are language models actually useful for time series forecasting? *Advances in Neural Information Processing Systems* 37 (2024), 60162–60191.

[32] Hua Tang, Chong Zhang, Mingyu Jin, Qinkai Yu, Zhenting Wang, Xiaobo Jin, Yongfeng Zhang, and Mengnan Du. 2025. Time series forecasting with llms: Understanding and enhancing model capabilities. *ACM SIGKDD Explorations Newsletter* 26, 2 (2025), 109–118.

[33] Minh-Tuan Tran, Xuan-May Le, Van-Nam Huynh, and Sung-Eui Yoon. 2024. PISD: A linear complexity distance beats dynamic time warping on time series classification and clustering. *Engineering Applications of Artificial Intelligence* 138 (2024), 109222.

[34] Shiyu Wang, Haixu Wu, Xiaoming Shi, Tengge Hu, Huakun Luo, Lintao Ma, James Y Zhang, and Jun Zhou. 2024. Timemixer: Decomposable multiscale mixing for time series forecasting. *arXiv preprint arXiv:2405.14616* (2024).

[35] Yihe Wang, Nan Huang, Taida Li, Yujun Yan, and Xiang Zhang. 2024. Medformer: A multi-granularity patching transformer for medical time-series classification. *Advances in Neural Information Processing Systems* 37 (2024), 36314–36341.

[36] Yuxuan Wang, Haixu Wu, Jiaxiang Dong, Guo Qin, Haoran Zhang, Yong Liu, Yunzhong Qiu, Jianmin Wang, and Mingsheng Long. 2024. Timexer: Empowering transformers for time series forecasting with exogenous variables. *Advances in Neural Information Processing Systems* 37 (2024), 469–498.

[37] Haixu Wu, Tengge Hu, Yong Liu, Hang Zhou, Jianmin Wang, and Mingsheng Long. 2022. Timesnet: Temporal 2d-variation modeling for general time series analysis. *arXiv preprint arXiv:2210.02186* (2022).

[38] Haixu Wu, Yao Xu, Jialin Wang, Guodong Long, and et al. 2021. Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[39] Haixu Wu, Hang Zhou, Mingsheng Long, and Jianmin Wang. 2023. Interpretable weather forecasting for worldwide stations with a unified deep model. *Nature Machine Intelligence* 5, 6 (2023), 602–611.

[40] Hao Xue and Flora D Salim. 2023. Promptcast: A new prompt-based learning paradigm for time series forecasting. *IEEE Transactions on Knowledge and Data Engineering* 36, 11 (2023), 6851–6864.

[41] Kyung Keun Yun, Sang Won Yoon, and Daehan Won. 2023. Interpretable stock price forecasting model using genetic algorithm-machine learning regressions and best feature subset selection. *Expert Systems with Applications* 213 (2023), 118803.

[42] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. 2023. Are transformers effective for time series forecasting?. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 37. 11121–11128.

[43] Jianqi Zhang, Jingyao Wang, Wenwen Qiang, Fanjiang Xu, and Changwen Zheng. 2026. Enhancing Large Language Models for Time-Series Forecasting via Vector-Injected In-Context Learning. *arXiv preprint arXiv:2601.07903* (2026).

[44] Yunhao Zhang and Junchi Yan. 2023. Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting. In *The eleventh international conference on learning representations*.

[45] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, and et al. 2021. Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting. *AAAI* 35, 12 (2021), 11106–11115.

[46] Tian Zhou, Peisong Niu, Liang Sun, Rong Jin, et al. 2023. One fits all: Power general time series analysis by pretrained lm. *Advances in neural information processing systems* 36 (2023), 43322–43355.


# A Datasets

We evaluate on diverse real-world forecasting benchmarks covering multiple domains and temporal resolutions. ETT contains electricity transformer measurements with four subsets (ETTh1/ETTh2 hourly, ETTm1/ETTm2 15-minute). Weather includes 21 meteorological variables sampled every 10 minutes, while ECL and Traffic

contain hourly electricity consumption and road occupancy data, respectively. We also evaluate on the widely used univariate M3 [22] and M4 [23] benchmarks spanning multiple domains and frequencies. Following prior long-term forecasting studies, all datasets use the same preprocessing and chronological train/validation/test splits to prevent information leakage. Dataset statistics are summarised in Table 6.

**Table 6: Detailed dataset statistics.**

| Dataset | Dim | Prediction Length | Dataset Size (Tr/Val/Te) | Frequency | Domain |
|---|---|---|---|---|---|
| ETTh1, ETTh2 | 7 | {96, 192, 336, 720} | (8545, 2881, 2881) | Hourly | Electricity |
| ETTm1, ETTm2 | 7 | {96, 192, 336, 720} | (34465, 11521, 11521) | 15 min | Electricity |
| Weather | 21 | {96, 192, 336, 720} | (36792, 5271, 10540) | 10 min | Weather |
| ECL | 321 | {96, 192, 336, 720} | (18317, 2633, 5261) | Hourly | Electricity |
| Traffic | 862 | {96, 192, 336, 720} | (12185, 1757, 3509) | Hourly | Transportation |
| M4-Yearly | 1 | 6 | (23000, 0, 23000) | Yearly | Demographic |
| M4-Quarterly | 1 | 8 | (24000, 0, 24000) | Quarterly | Finance |
| M4-Monthly | 1 | 18 | (48000, 0, 48000) | Monthly | Industry |
| M4-Weekly | 1 | 13 | (359, 0, 359) | Weekly | Macro |
| M4-Daily | 1 | 14 | (4227, 0, 4227) | Daily | Micro |
| M4-Hourly | 1 | 48 | (414, 0, 414) | Hourly | Other |
| M3-Yearly | 1 | 6 | (645, 0, 645) | Yearly | Demographic |
| M3-Quarterly | 1 | 8 | (756, 0, 756) | Quarterly | Finance |
| M3-Monthly | 1 | 18 | (1428, 0, 1428) | Monthly | Industry |
| M3-Others | 1 | 8 | (174, 0, 174) | Weekly | Macro |

## B Evaluation Metrics

Let $H$ denote the prediction horizon, and let $Y_h$ and $\hat{Y}_h$ be the ground-truth and predicted value at step $h \in \{1, \ldots, H\}$. We compute:

$$\text{sMAPE} = \frac{200}{H} \sum_{h=1}^{H} \frac{\left|Y_h - \hat{Y}_h\right|}{|Y_h| + \left|\hat{Y}_h\right|}, \tag{21}$$

For MASE, we normalise the absolute error by the in-sample seasonal naive error:

$$\text{MASE} = \frac{1}{H} \sum_{h=1}^{H} \frac{\left|Y_h - \hat{Y}_h\right|}{\frac{1}{H-s}\sum_{j=s+1}^{H}\left|Y_j - Y_{j-s}\right|}, \tag{22}$$

where $s$ is the seasonality (periodicity) of the series.

Finally, OWA combines the relative sMAPE and MASE compared to the M4 Naive2 baseline:

$$\text{OWA} = \frac{1}{2}\left(\frac{\text{sMAPE}}{\text{sMAPE}_{\text{Naive2}}} + \frac{\text{MASE}}{\text{MASE}_{\text{Naive2}}}\right). \tag{23}$$

## C Further Ablation Studies

**Parameters $\lambda$ and $\beta$.** We sweep $\lambda, \beta \in \{0.5, 1, 1.5, 2\}$ on ETTh1 and ECL with all other settings fixed. As Table 7 shows, results are stable across all combinations, and the default $(\lambda, \beta) = (1, 1)$ achieves the best or tied-best performance (ETTh1: 0.376/0.408; ECL: 0.154/0.245 MSE/MAE).

**Patch length $\ell$.** Table 8 reports results for $\ell \in \{6, 12, 16, 24\}$, each paired with an appropriate scale schedule. Performance is stable for smaller patches, but degrades with coarser tokenisation ($\ell \geq 16$), likely due to loss of fine-grained temporal detail. We adopt $\ell$=12 as the default.

**Scale Token Schedule.** Table 9 evaluates schedules with 1–5 refinement steps. Accuracy improves consistently up to 4 steps, with gains saturating beyond that. The 4-step schedule $\{1, 2, 4, 8\}$ achieves the best results on both datasets and is used as default; extending to 5 steps offers no further benefit.

| ETTh1 MSE↓ | | | | | ETTh1 MAE↓ | | | | |
|---|---|---|---|---|---|---|---|---|---|
| | $\beta$=0.5 | $\beta$=1 | $\beta$=1.5 | $\beta$=2 | | $\beta$=0.5 | $\beta$=1 | $\beta$=1.5 | $\beta$=2 |
| $\lambda$=0.5 | 0.378 | 0.377 | 0.379 | 0.381 | $\lambda$=0.5 | 0.411 | 0.410 | 0.410 | 0.411 |
| $\lambda$=1 | **0.377** | **0.376** | 0.377 | 0.381 | $\lambda$=1 | 0.409 | **0.408** | 0.410 | 0.411 |
| $\lambda$=1.5 | 0.378 | 0.377 | 0.379 | 0.379 | $\lambda$=1.5 | 0.409 | 0.409 | **0.408** | 0.410 |
| $\lambda$=2 | 0.379 | 0.381 | 0.378 | 0.379 | $\lambda$=2 | 0.411 | 0.409 | 0.409 | 0.411 |
| **ECL MSE↓** | | | | | **ECL MAE↓** | | | | |
| | $\beta$=0.5 | $\beta$=1 | $\beta$=1.5 | $\beta$=2 | | $\beta$=0.5 | $\beta$=1 | $\beta$=1.5 | $\beta$=2 |
| $\lambda$=0.5 | 0.156 | **0.154** | 0.155 | 0.158 | $\lambda$=0.5 | 0.246 | 0.246 | 0.247 | 0.248 |
| $\lambda$=1 | 0.156 | **0.154** | 0.156 | 0.158 | $\lambda$=1 | 0.246 | **0.245** | 0.248 | 0.247 |
| $\lambda$=1.5 | 0.155 | 0.155 | 0.155 | 0.157 | $\lambda$=1.5 | **0.245** | 0.247 | 0.246 | 0.248 |
| $\lambda$=2 | 0.158 | 0.157 | 0.157 | 0.158 | $\lambda$=2 | 0.248 | 0.247 | 0.248 | 0.248 |

**Table 7: Sensitivity to $\lambda$ and $\beta$ (MSE/MAE↓). Rows vary $\lambda$, columns vary $\beta$, in $\{0.5, 1, 1.5, 2\}$.**

| Dataset | $\ell$=6 | $\ell$=12 | $\ell$=16 | $\ell$=24 |
|---|---|---|---|---|
| ETTh1 (MSE↓) | 0.377 | **0.376** | 0.379 | 0.381 |
| ETTh1 (MAE↓) | 0.409 | **0.408** | 0.412 | 0.414 |
| ECL (MSE↓) | 0.156 | **0.154** | 0.158 | 0.161 |
| ECL (MAE↓) | 0.247 | **0.245** | 0.248 | 0.259 |

**Table 8: Patch length ablation. Scale schedules: $\ell$=6: $\{1, 2, 4, 8\}$, $\ell$=12: $\{2, 4, 8, 16\}$, $\ell$=16: $\{1, 2, 4, 6\}$, $\ell$=24: $\{1, 2, 3, 4\}$.**

| Steps | Scale List | ETTh1-MSE↓ | ETTh1-MAE↓ | ECL-MSE↓ | ECL-MAE↓ |
|---|---|---|---|---|---|
| 1 | {8} | 0.381 | 0.412 | 0.158 | 0.248 |
| 2 | {4, 8} | 0.379 | 0.410 | 0.157 | 0.247 |
| | {2, 8} | 0.379 | 0.411 | 0.157 | 0.247 |
| | {1, 8} | 0.380 | 0.409 | 0.157 | 0.247 |
| 3 | {2, 4, 8} | 0.380 | 0.411 | 0.158 | 0.248 |
| | {1, 4, 8} | 0.378 | 0.409 | 0.158 | 0.247 |
| | {3, 6, 8} | 0.379 | 0.410 | 0.156 | 0.248 |
| 4 | {1, 2, 4, 8} | **0.376** | **0.408** | **0.154** | **0.245** |
| | {1, 3, 6, 8} | 0.379 | 0.409 | 0.159 | 0.247 |
| 5 | {1, 2, 4, 6, 8} | 0.379 | 0.410 | 0.159 | 0.248 |

**Table 9: Scale token schedule ablation. The 4-step schedule $\{1, 2, 4, 8\}$ performs best and is used as default.**

**Effect of the Refine Loss** The refine loss $\mathcal{L}_{\text{refine}}$ propagates gradients into the shared encoder and decoder, aligning token representations at each scale with their downscaled targets. As Table 10 shows, removing it degrades accuracy across all datasets. Full LLM backbone fine-tuning yields only marginal gains at 15.21× the training cost, confirming that scale-wise supervision via $\mathcal{L}_{\text{refine}}$ is both effective and efficient.

| Method | ETTh1 | ECL | Weather | Traffic | Train Time |
|---|---|---|---|---|---|
| SCALER (default) | **0.376** | **0.154** | **0.217** | **0.367** | 1.00× |
| w/o refine loss | 0.380 | 0.158 | 0.220 | 0.371 | 1.00× |
| w/ LLM backbone fine-tuning | 0.376 | 0.152 | 0.218 | 0.368 | 15.21× |

**Table 10: Effect of refine loss and LLM fine-tuning (MSE↓).**

**LLM Refiner vs. Smaller Model Refiners** We replace the LLM refiner with PatchTST [28] and iTransformer [20], keeping all else unchanged. Both alternatives outperform their standalone counterparts, confirming the value of the coarse-to-fine structure. However, LLaMA-7B consistently achieves the best accuracy (Table 12), indicating that the LLM's pretrained generative prior provides additional coherence that smaller models cannot match.

**Table 11: Full long-term forecasting results.**

| | | Test-Time Scaling | | | | LLM-based Model | | | | | | | | | | | | Other Deep Learning Model | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Dataset | Length | SCALER (Ours) | | TimeReasoner | | LVICL | | AutoTimes | | TimeLLM | | FPT | | Chronos | | TimeFM | | iTransformer | | DLinear | | PatchTST | | TimesNet | |
| | | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE |
| ETTh1 | 96 | **0.345** | **0.386** | 0.36 | 0.39 | 0.351 | 0.389 | 0.36 | 0.4 | 0.362 | 0.392 | 0.376 | 0.397 | 0.369 | 0.409 | 0.384 | 0.426 | 0.386 | 0.405 | 0.375 | 0.399 | 0.37 | 0.399 | 0.384 | 0.402 |
| | 192 | **0.374** | **0.406** | 0.387 | 0.412 | 0.379 | 0.408 | 0.388 | 0.419 | 0.398 | 0.418 | 0.416 | 0.418 | 0.399 | 0.429 | 0.415 | 0.447 | 0.422 | 0.439 | 0.405 | 0.416 | 0.413 | 0.421 | 0.557 | 0.436 |
| | 336 | **0.385** | **0.413** | 0.425 | 0.417 | 0.392 | 0.417 | 0.401 | 0.429 | 0.43 | 0.427 | 0.442 | 0.433 | 0.413 | 0.439 | 0.429 | 0.456 | 0.444 | 0.457 | 0.439 | 0.443 | 0.422 | 0.436 | 0.491 | 0.469 |
| | 720 | **0.399** | **0.425** | 0.439 | 0.449 | 0.402 | 0.434 | 0.406 | 0.44 | 0.442 | 0.457 | 0.477 | 0.456 | 0.423 | 0.457 | 0.44 | 0.475 | 0.5 | 0.498 | 0.472 | 0.49 | 0.447 | 0.466 | 0.521 | 0.5 |
| | Avg | **0.376** | **0.408** | 0.403 | 0.417 | 0.381 | 0.412 | 0.389 | 0.422 | 0.408 | 0.423 | 0.427 | 0.426 | 0.401 | 0.434 | 0.417 | 0.451 | 0.438 | 0.45 | 0.423 | 0.437 | 0.413 | 0.431 | 0.458 | 0.45 |
| ETTh2 | 96 | **0.265** | **0.33** | 0.283 | 0.336 | 0.27 | 0.332 | 0.282 | 0.342 | 0.288 | 0.341 | 0.287 | 0.341 | 0.271 | 0.333 | 0.277 | 0.34 | 0.304 | 0.36 | 0.289 | 0.353 | 0.274 | 0.336 | 0.34 | 0.374 |
| | 192 | 0.327 | 0.376 | 0.346 | 0.388 | 0.333 | 0.378 | 0.348 | 0.387 | 0.351 | 0.389 | 0.35 | 0.383 | **0.326** | **0.371** | 0.334 | 0.38 | 0.377 | 0.403 | 0.383 | 0.418 | 0.339 | 0.379 | 0.402 | 0.414 |
| | 336 | **0.322** | **0.371** | 0.352 | 0.398 | 0.325 | 0.377 | 0.365 | 0.412 | 0.362 | 0.401 | 0.373 | 0.395 | 0.338 | 0.394 | 0.34 | 0.393 | 0.405 | 0.429 | 0.448 | 0.465 | 0.329 | 0.38 | 0.452 | 0.452 |
| | 720 | **0.368** | 0.415 | 0.411 | 0.432 | 0.375 | 0.416 | 0.412 | 0.44 | 0.415 | 0.44 | 0.401 | 0.443 | 0.372 | **0.409** | 0.384 | 0.426 | 0.443 | 0.464 | 0.605 | 0.551 | 0.379 | 0.422 | 0.462 | 0.468 |
| | Avg | **0.321** | **0.373** | 0.348 | 0.389 | 0.326 | 0.376 | 0.352 | 0.395 | 0.354 | 0.393 | 0.353 | 0.391 | 0.327 | 0.377 | 0.334 | 0.385 | 0.382 | 0.414 | 0.431 | 0.446 | 0.33 | 0.379 | 0.414 | 0.427 |
| ETTm1 | 96 | 0.265 | 0.342 | 0.281 | **0.337** | 0.269 | 0.343 | 0.274 | 0.343 | 0.284 | 0.341 | 0.301 | 0.343 | **0.264** | **0.337** | 0.28 | 0.357 | 0.312 | 0.366 | 0.299 | 0.343 | 0.29 | 0.342 | 0.338 | 0.375 |
| | 192 | **0.302** | 0.368 | 0.32 | 0.368 | 0.309 | 0.371 | 0.316 | 0.37 | 0.327 | 0.363 | 0.348 | **0.362** | 0.303 | 0.364 | 0.321 | 0.386 | 0.347 | 0.385 | 0.335 | 0.365 | 0.332 | 0.369 | 0.374 | 0.387 |
| | 336 | 0.341 | 0.386 | 0.363 | **0.379** | 0.343 | 0.39 | 0.344 | 0.39 | 0.368 | 0.387 | 0.386 | 0.397 | **0.337** | 0.383 | 0.357 | 0.405 | 0.379 | 0.404 | 0.369 | 0.386 | 0.366 | 0.392 | 0.41 | 0.411 |
| | 720 | 0.388 | **0.398** | 0.413 | 0.416 | 0.391 | 0.408 | 0.392 | 0.418 | 0.423 | 0.419 | 0.43 | 0.427 | **0.384** | 0.401 | 0.406 | 0.424 | 0.441 | 0.442 | 0.425 | 0.421 | 0.416 | 0.42 | 0.478 | 0.45 |
| | Avg | 0.324 | 0.374 | 0.344 | 0.373 | 0.328 | 0.378 | 0.332 | 0.38 | 0.35 | 0.378 | 0.366 | 0.382 | **0.322** | **0.371** | 0.341 | 0.393 | 0.37 | 0.399 | 0.357 | 0.378 | 0.351 | 0.38 | 0.4 | 0.406 |
| ETTm2 | 96 | **0.148** | **0.245** | 0.157 | 0.248 | 0.155 | 0.246 | 0.159 | 0.25 | 0.168 | 0.25 | 0.172 | 0.258 | 0.159 | 0.252 | 0.157 | 0.249 | 0.179 | 0.271 | 0.167 | 0.269 | 0.165 | 0.255 | 0.187 | 0.267 |
| | 192 | **0.204** | **0.281** | 0.208 | 0.291 | 0.21 | 0.282 | 0.213 | 0.285 | 0.216 | 0.294 | 0.231 | 0.287 | 0.215 | 0.288 | 0.212 | 0.285 | 0.242 | 0.313 | 0.224 | 0.303 | 0.22 | 0.292 | 0.249 | 0.309 |
| | 336 | **0.253** | **0.311** | 0.257 | 0.319 | 0.258 | 0.317 | 0.261 | 0.323 | 0.269 | 0.323 | 0.282 | 0.329 | 0.264 | 0.324 | 0.261 | 0.32 | 0.288 | 0.344 | 0.281 | 0.342 | 0.274 | 0.329 | 0.321 | 0.351 |
| | 720 | **0.332** | **0.372** | 0.357 | 0.384 | 0.335 | 0.378 | 0.341 | 0.381 | 0.363 | 0.387 | 0.373 | 0.387 | 0.343 | 0.387 | 0.338 | 0.382 | 0.378 | 0.397 | 0.397 | 0.421 | 0.362 | 0.385 | 0.408 | 0.403 |
| | Avg | **0.234** | **0.302** | 0.245 | 0.311 | 0.239 | 0.306 | 0.243 | 0.31 | 0.254 | 0.314 | 0.265 | 0.315 | 0.245 | 0.313 | 0.242 | 0.309 | 0.272 | 0.331 | 0.267 | 0.333 | 0.255 | 0.315 | 0.291 | 0.333 |
| ECL | 96 | **0.12** | **0.222** | 0.128 | **0.222** | 0.126 | 0.224 | 0.129 | 0.225 | 0.131 | 0.224 | 0.139 | 0.238 | 0.131 | 0.233 | 0.129 | 0.23 | 0.132 | 0.227 | 0.153 | 0.237 | 0.129 | **0.222** | 0.168 | 0.272 |
| | 192 | **0.143** | **0.231** | 0.146 | 0.24 | 0.148 | 0.235 | 0.147 | 0.241 | 0.152 | 0.241 | 0.153 | 0.251 | 0.154 | 0.244 | 0.152 | 0.241 | 0.153 | 0.249 | 0.152 | 0.249 | 0.147 | 0.24 | 0.184 | 0.289 |
| | 336 | 0.158 | 0.246 | **0.153** | **0.242** | 0.161 | 0.252 | 0.162 | 0.258 | 0.16 | 0.248 | 0.169 | 0.266 | 0.167 | 0.262 | 0.165 | 0.259 | 0.167 | 0.262 | 0.169 | 0.267 | 0.163 | 0.259 | 0.198 | 0.3 |
| | 720 | 0.194 | **0.281** | **0.189** | 0.294 | 0.196 | 0.282 | 0.199 | 0.288 | 0.192 | 0.298 | 0.206 | 0.297 | 0.204 | 0.293 | 0.201 | 0.29 | 0.254 | 0.335 | 0.233 | 0.344 | 0.197 | 0.29 | 0.22 | 0.32 |
| | Avg | **0.154** | **0.245** | **0.154** | 0.25 | 0.158 | 0.248 | 0.159 | 0.253 | 0.159 | 0.253 | 0.167 | 0.263 | 0.164 | 0.258 | 0.162 | 0.255 | 0.161 | 0.256 | 0.177 | 0.274 | 0.159 | 0.253 | 0.192 | 0.295 |
| Weather | 96 | 0.141 | **0.193** | **0.136** | 0.198 | 0.146 | 0.196 | 0.153 | 0.203 | 0.147 | 0.201 | 0.162 | 0.212 | 0.145 | 0.198 | 0.151 | 0.202 | 0.163 | 0.211 | 0.152 | 0.237 | 0.149 | 0.198 | 0.172 | 0.22 |
| | 192 | 0.186 | 0.238 | **0.181** | **0.228** | 0.19 | 0.24 | 0.201 | 0.25 | 0.189 | 0.234 | 0.204 | 0.248 | 0.188 | 0.242 | 0.196 | 0.248 | 0.205 | 0.25 | 0.22 | 0.282 | 0.194 | 0.241 | 0.219 | 0.261 |
| | 336 | **0.234** | **0.277** | 0.258 | **0.277** | 0.24 | 0.28 | 0.256 | 0.293 | 0.262 | 0.279 | 0.254 | 0.286 | 0.238 | 0.281 | 0.248 | 0.289 | 0.254 | 0.289 | 0.265 | 0.319 | 0.245 | 0.282 | 0.28 | 0.306 |
| | 720 | 0.306 | 0.32 | **0.296** | **0.312** | 0.312 | 0.33 | 0.331 | 0.345 | 0.304 | 0.316 | 0.326 | 0.337 | 0.309 | 0.331 | 0.322 | 0.34 | 0.329 | 0.34 | 0.323 | 0.362 | 0.314 | 0.334 | 0.365 | 0.359 |
| | Avg | **0.217** | 0.257 | 0.218 | **0.254** | 0.224 | 0.262 | 0.235 | 0.273 | 0.225 | 0.257 | 0.237 | 0.27 | 0.22 | 0.263 | 0.229 | 0.27 | 0.238 | 0.272 | 0.24 | 0.3 | 0.226 | 0.264 | 0.259 | 0.287 |
| Traffic | 96 | **0.334** | **0.244** | 0.357 | 0.247 | 0.338 | 0.245 | 0.343 | 0.248 | 0.362 | 0.248 | 0.388 | 0.282 | 0.343 | 0.249 | 0.35 | 0.253 | 0.351 | 0.257 | 0.41 | 0.282 | 0.36 | 0.249 | 0.593 | 0.321 |
| | 192 | **0.358** | 0.25 | 0.37 | 0.248 | 0.36 | 0.251 | 0.362 | 0.257 | 0.374 | **0.247** | 0.407 | 0.29 | 0.366 | 0.255 | 0.372 | 0.26 | 0.364 | 0.265 | 0.423 | 0.287 | 0.379 | 0.256 | 0.617 | 0.333 |
| | 336 | **0.378** | **0.254** | 0.381 | 0.264 | 0.382 | 0.259 | 0.379 | 0.266 | 0.385 | 0.271 | 0.412 | 0.294 | 0.388 | 0.263 | 0.395 | 0.268 | 0.382 | 0.273 | 0.436 | 0.296 | 0.392 | 0.264 | 0.629 | 0.333 |
| | 720 | **0.399** | **0.276** | 0.421 | 0.281 | 0.401 | 0.285 | 0.413 | 0.284 | 0.43 | 0.288 | 0.45 | 0.312 | 0.407 | 0.289 | 0.415 | 0.295 | 0.42 | 0.292 | 0.466 | 0.315 | 0.432 | 0.286 | 0.64 | 0.33 |
| | Avg | **0.367** | **0.256** | 0.382 | 0.259 | 0.37 | 0.26 | 0.374 | 0.264 | 0.388 | 0.264 | 0.414 | 0.294 | 0.376 | 0.264 | 0.383 | 0.269 | 0.379 | 0.272 | 0.434 | 0.295 | 0.391 | 0.264 | 0.62 | 0.336 |
| | 1st Count | **19** | **19** | 5 | 7 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 4 | 4 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |

| **Stage** | **ETTh1 (MSE↓)** | **ECL (MSE↓)** |
|---|---|---|
| Stage-1 (coarse shape only) | 0.391 | 0.168 |
| Stage-2 | 0.387 | 0.165 |
| Stage-3 | 0.384 | 0.162 |
| SCALER (Stage-4, default) | **0.376** | **0.154** |

**Table 13: Intermediate-stage MSE. Performance improves monotonically, validating each refinement step.**

| **Stage-II Refiner** | **ETTh1** | | **ECL** | |
|---|---|---|---|---|
| | MSE↓ | MAE↓ | MSE↓ | MAE↓ |
| SCALER (LLaMA-7B, default) | **0.376** | **0.408** | **0.154** | **0.245** |
| SCALER (PatchTST) | 0.401 | 0.425 | 0.159 | 0.254 |
| SCALER (iTransformer) | 0.424 | 0.443 | 0.158 | 0.251 |

**Table 12: LLM vs. smaller refiners. LLaMA-7B outperforms both alternatives, showing the pretrained prior contributes beyond the coarse-to-fine structure alone.**

# D Further Analysis

**Intermediate-Stage Analysis** Table 13 shows that MSE decreases monotonically from Stage 1 (coarse shape) through Stage 4 (full refinement) on both ETTh1 and ECL, confirming that each refinement step contributes meaningfully rather than redundantly.

**Discussion: downscaling and aliasing.** The normalise-then-interpolate operator risks aliasing if the downsampling grid coincides with zeros of a periodic component. The default 8× ratio ($T$=96→12) retains sufficient structure for our benchmarks, but for high-frequency signals a smaller ratio (4× or 6×) is advisable. Adaptive ratio selection and explicit trend/seasonality encoding are left for future work.

# E Full Long-Term Forecasting Results

Table 11 reports full results on seven benchmarks across horizons {96, 192, 336, 720}. SCALER achieves consistently strong performance, particularly at longer horizons, winning 19 of 28 dataset–horizon pairs on both MSE and MAE. Baseline results are taken from [43]; TimeReasoner is rerun under our setup.